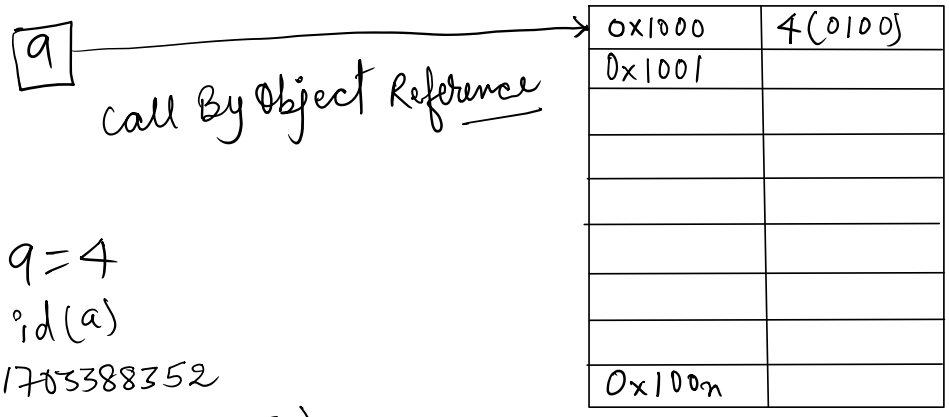
Variable And Memory References




```
>>> a=4
>>> id(a)
   1703388352
>>> hex(1703388352)
   '0x6587a4c0'
>>> id(4)
   1703388352
```

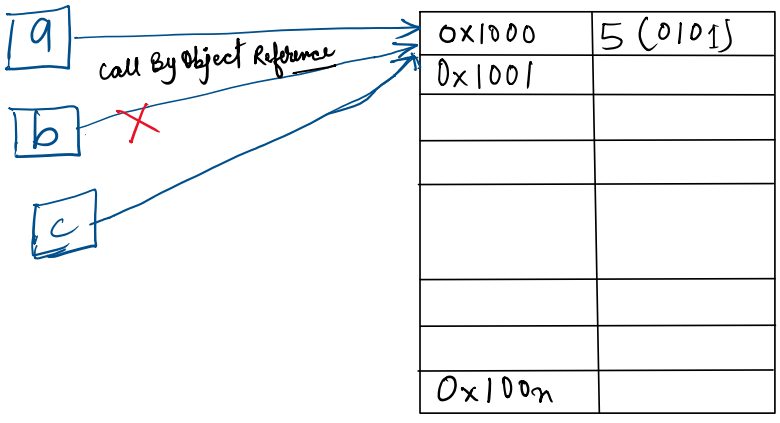
Aliasing

Aliasing in Python occurs when two or more variables refer to the exact same object in memory.
Because Python uses references to bind names to objects, assigning one variable to another (b = a) does not copy the data—it merely creates a new reference (an "alias") to the existing data.

Case-1

```
>>> a = 5
>>> b = a
>>> id(a)
   1703388384
>>> id(b)
   1703388384
>>> c = b
>>> id(c)
   1703388384

>>> del b
>>> b
Name Error: name 'b' is not defined.
```

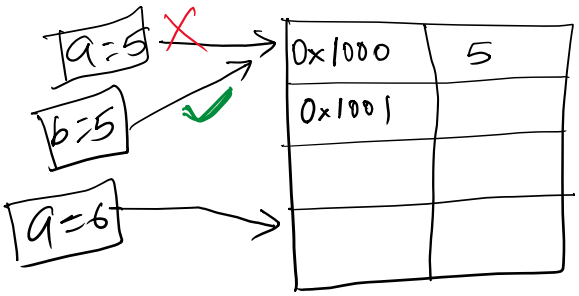


```
>>> c ↵
5
```

case-2

```
>>> a = 5
>>> b = a
>>> a = 6
```

b = ?



Reference Counting

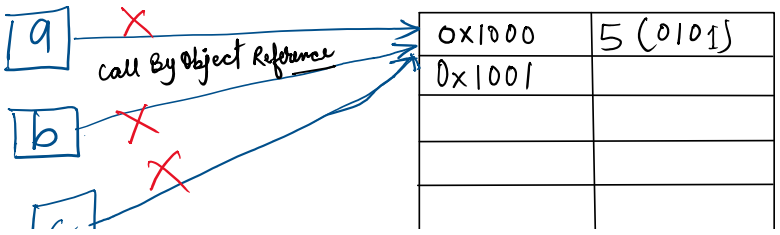
Reference counting is the primary memory management mechanism used by Python's standard implementation (CPython) to automatically track and manage application memory. Every time you create an object, Python tracks how many variables, data structures, or functions are actively pointing to that specific object.

```
>>> import sys
>>> a = "Bihar" ↵
>>> b = a ↵
>>> c = b ↵
>>> id(a) ↵
   1872263895000
>>> id(b) ↵
   1872263895000
>>> id(c) ↵
   1872263895000
>>> sys.getrefcount(a) ↵
   4
```

Garbage Collection

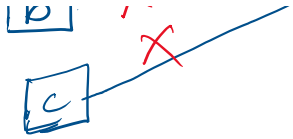
Garbage collection in Python is an automated memory management process that deletes unused objects to free up heap space. Unlike languages like C or C++, where you must manually allocate and release memory, Python handles this behind the scenes.

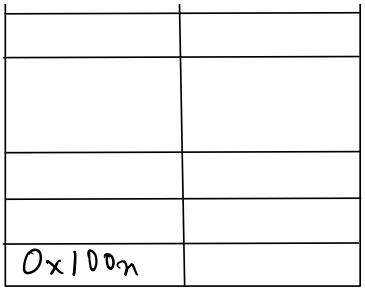
```
>>> del a
>>> del b
>>> del c
```



>>> del c

Garbage collector





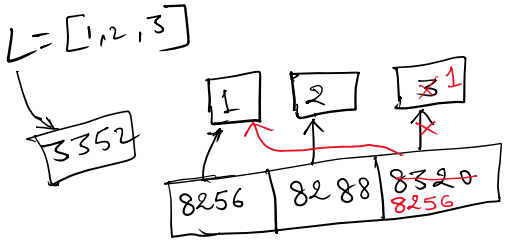
Weird Stuff

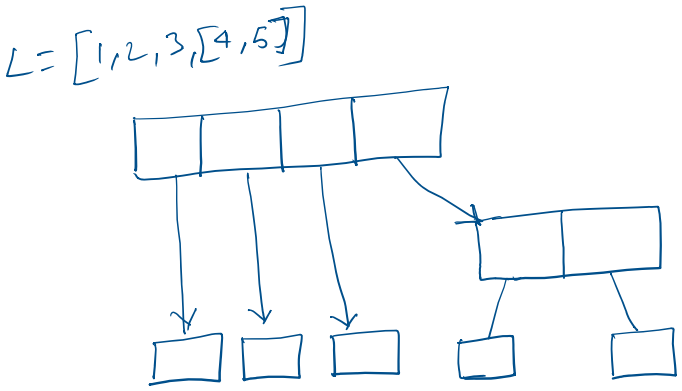
```
>>> a = 2
>>> b = a
>>> c = b
>>> sys.getrefcount(a)
373
```

sys.getrefcount() fluctuates because **passing an object as an argument into the function creates a temporary reference**. This adds exactly 1 to the actual count during execution. The exact output varies based on where the code runs, variable assignments, and CPython's memory caching.

List Memory References

```
>>> L=[1,2,3]
>>> id(L)
187226379352
>>> id(1)
1703388256
>>> id(L[0])
1703388256
>>> id(2)
1703388288
>>> id(3)
1703388320
>>> L[2]=1
>>> L
[1,2,1]
>>> id(L[2])
1703388256
>>> L1=[1,2,3,[4,5]]
```





1703388256
>>> L1 = [1,2,3,[4,5]]

Mutability

In Python, **mutability refers to the capability of an object to change its internal state or data in place** after it has been created.

If an object is **mutable**, you can modify, add, or delete its contents without changing its unique memory identity (id()). If an object is **immutable**, its value cannot be altered; any operation that seems to change it actually creates a brand-new object elsewhere in memory.

0x1000	5 (0101)
0x1001	
0x100n	

→ Mutability refers to the ability to change or edit data in it's memory location.

Mutability depends on the data type.

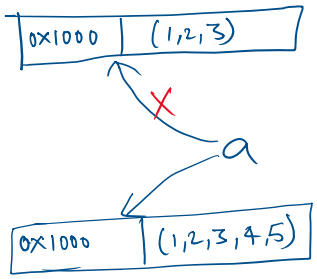
Immutable Data Types	Mutable Data Types
String	
int	
float	List
Bool	Dictionary
complex	Sets
Tuple	

String

```
>>> a = "Hello"
>>> id(a)
   94720
>>> a = a + "World"
>>> a
   HelloWorld
>>> id(a)
   02992
```

Tuple

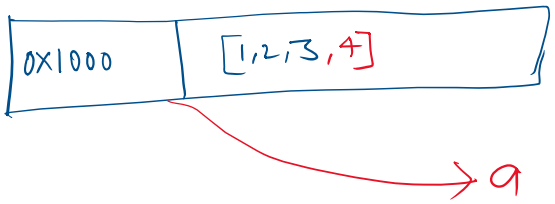
```
>>> T = (1,2,3)
>>> id(T)
   26336
>>> T = T + (4,5)
>>> T
   (1,2,3,4,5)
>>> id(T)
   12136
```



List

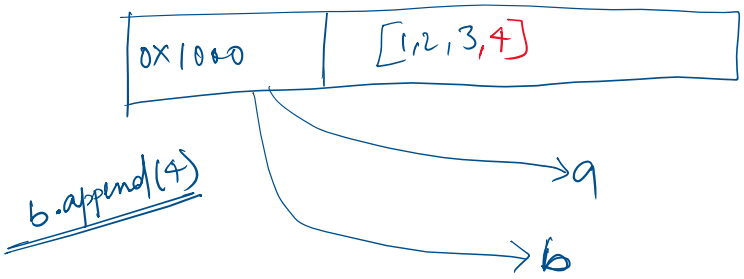
List

```
>>> L=[1,2,3]
>>> id(L)
  04680
>>> L.append(4)
>>> L
  [1,2,3,4]
>>> id(L)
  04680
```



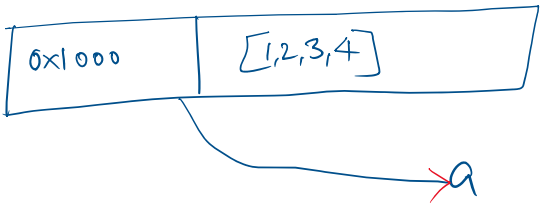
Side effects of Mutation

```
>>> L=[1,2,3]
>>> L1=L
>>> L1
  [1,2,3]
>>> L
  [1,2,3]
>>> id(L)
  38088
>>> id(L1)
  38088
>>> L1.append(4)
>>> L1
  [1,2,3,4]
>>> id(L1)
  38088
>>> L1
  [1,2,3,4]
>>> L
  [1,2,3,4]
```



cloning

```
>>> L=[1,2,3,4]
>>> L1=L[:]
>>> id(L)
  38088
>>> id(L1)
```



```
38088
>>> id(L1)
04680
>>> L1.append(5)
>>> L1
[1,2,3,4,5]
>>> L
[1,2,3,4]
```